scientific reports

Check for updates

OPEN

A new advance on dimensional-aware scalar, vector and matrix operations in C++

Eduard George Stan[1,2], Dan Andrei Ciubotaru[1], Michele Renda[1] & Călin Alexa[1]✉

We review the dimensional check problem of the high-level programming languages, discuss the existing solutions, and come up with a new solution suited for scientific and engineering computations. Then, we introduce `Univec`, our `C++` library designed to make scalar, vector, and matrix operations using units of measurement. Moreover, `Univec` supports dimensional-aware operations for complex numbers, quaternions, octonions, and sedenions. We provide tables of the relevant functions and operators implemented. Our library was compared with several existing solutions, and the results are shown in the performance section. Finally, we present our future plans for improving the current implementation.

Dimensional analysis is one of the most powerful tools available in physics to verify the correctness of mathematical formulas describing physical processes. However, this tool alone is not able to spot all possible mistakes. It allows only a fast check, highlights faulty operations, and provides a consistent examination of the whole process.

Nowadays, complex simulations and derivations are performed using computing algorithms taking advantage of high-speed calculation and numerical correctness due to the advances in computing technology. However, when using complex algorithms spanning thousands of source code lines divided into tens of files, we tend not to automatically trust the results or, much worse, accept them blindly. Moreover, because there is no bug-free software implementation, we have to mitigate the impact of possible errors using countless checks in code and proper testing procedures, which are often as time-consuming as the development itself.

Therefore, an appropriate approach is to use techniques that could help reduce the overall cost of software development[1]. Unfortunately, due to historical reasons, most high-level programming languages lack a dimensional check as a language feature; they work on floats and integers, leaving to the programmer's judgement the responsibility to keep track of their semantic meaning, leading to many potential mistakes.

Several libraries provide dimensional analysis check at compile time or at run-time. However, as shown by McKeever[2], there is an evident reticence to adopt such solutions due to the following reasons:

- Difficulty in using unfamiliar and complex libraries
- Fear of adding external dependencies
- Concerns about the performance impact
- Possible limitations on using external libraries or complex data structures such as matrices and vectors.

In "Existing solutions", we review the existing solutions available in the most common programming languages. We describe our proposed solution and present concrete examples in "Our implementation". In "Unit testing", we discuss the testing approach, and in "Performance study", we provide a performance analysis. Limitations of the current development, along with plans for improvements are addressed in "Limitations and future plans", and conclusions are drawn in "Conclusions".

Existing solutions

A comprehensive review of several existing solutions for dimensional analysis in programming languages can be found in Preussner's work[3]. Apple's Swift[4] and `F#`[5] are two widespread programming languages that provide native support for UoM-aware (units of measure) quantities. Promoting UoM analysis to a language feature has the benefit of immediate access to meaningful error messages. Unfortunately, only a few programming languages offer native support for this important topic. Instead, most languages have an external library that relies on generic programming techniques.

[1]IFIN-HH, Particle Physics Department, Măgurele 077125, Romania. [2]Faculty of Physics, University of Bucharest, Măgurele 077125, Romania. ✉email: calin.alexa@nipne.ro

The most popular solution for C++ is Boost.Units[6], which makes possible the UoM validation at compile time. Its main advantage is that it is included in the Boost framework, a comprehensive set of libraries that are used as a launchpad for features that are later included in the language's standard library. Unfortunately, Boost is quite often seen as a heavy dependency. For the FORTRAN language there is the PHYSUNITS module[7] which provides support for dimensional-aware routine creation. While this solution is versatile, allowing working with both F77 and F90 code, because the dimension information is carried alongside numerical values, PHYSUNITS introduces a run-time execution overhead in terms of speed and memory usage. Dimensional analysis is not limited to compiled languages, Python supports run-time dimensional analysis checks thanks to libraries like Pint[8].

Almost all existing libraries do not provide native support for vectorial computation. However, for vectorial operations in high-energy physics there are two solutions: CLHEP[9] and Eigen[10]. They provide representations for both 2D and 3D vectorial quantities and n-dimensional matrices, but none of them has built-in support for dimensional-aware units, relying only on primitive numeric types available in C++.

We have to mention that our approach is not the first attempt to solve this specific issue. For example, the CORSIKA 8 simulation tool[11], a C++ rewrite of CORSIKA[12], uses a similar solution to address the same problem of combining unit systems with vector arithmetic. The approach adopted there was to combine Eigen v3[10] with the PhysUnits library[13], allowing them to perform dimensional-aware vector arithmetic without any performance penalty[11].

Our implementation

To improve the accuracy and lower the run-time overhead of the scientific and engineering computations, we developed Univec, our C++ library that allows scalar, vector, matrix, complex, quaternion, octonions, and sedenion operations integrating the Boost.Units[6] library features.

We decided to implement a library that uses an existing UoM analysis library and wrap it in an easy-to-use interface for physicists. We rely on Boost::Units for dimensional checks, but we provide classes that can represent *n*-dimensional vectors and matrices. This way, we can provide dimensionally correct methods for common operations such as vector norms and products, matrix determinants and other operations. The performance of our solution is in line with Boost::Units, because one of our development objectives was not to add any perceptible runtime overhead and to make possible automatic optimizations by the compiler.

A summary of the functions and aliases available for vector and matrix types can be found in Tables 1, 2, 3, 4, 5, 6 and 7, while the complete source code, released under LGPLv3 license, can be found at the address https://gitlab.com/micrenda/univec.

All the methods implemented return a copy of the original vector. However, for methods for which the result is compatible with the calling class type, we created an in-place version of the method, which modifies the object instance. These methods start with the prefix do (e.g. doConj()) to be easily differentiated from the copy version of the same method. These methods are usually faster than those that copy data because they do not allocate additional memory for their operations.

Our implementation heavily uses generic template programming, shifting the cost of the dimensional check at compiling time and removing, in principle, any run-time overhead. However, the drawback is an increased usage complexity, which may discourage non-experienced C++ developers. We tried to mitigate this issue by providing custom descriptive error messages and a set of default quantity aliases covering the most common quantities used in physics. This approach allows easy-to-remember type names when defining a variable, such as:

```
dfpe::QtySiVelocity my_var;
```

which is a much shorter name than its canonical name:

```
boost::units::quantity<boost::units::si::velocity>  my_var;
```

.

We decided to keep the set of aliases in SI, GCS and Gauss units, isolated from the main library development and were published as a separated project, at the address https://gitlab.com/micrenda/qtydef.

Vector operations

Univec allows writing a compact and semantically clear code with dimensional-check at compile-time. This is done using a set of classes, shown in Table 5, representing different types of quantities, such as vectors and matrices. In our implementation we tried to prioritize clarity and performance, two objectives that are difficult to reconcile because fast code tends to be quite complex and difficult to read. We implemented all the *n*-dimensional Cartesian vector classes, including complex numbers and quaternions, in a single header-only class. This approach required heavy use of template programming. However, it provides two important benefits: code duplication was significantly reduced, and we got a highly optimized implementation because the method resolution is performed at compile-time since we do not have inheritance and virtual functions, allowing efficient compiler optimizations. Furthermore, the complexity of the code due to generic programming was mitigated using the last C++ keywords like requires, which improve substantially the code readability.

During the design, we realized that complex numbers, quaternions, octonions, and sedenions methods were a super-set of the methods available in Cartesian *n*—dimensional vectors (see Table 2). Therefore, we used the

Operation	Method		Note
$\mathbf{v} \cdot \mathbf{u}$	`v.dot()`	Dot product	
$\mathbf{v} \times \mathbf{u}$	`v.cross(u)`	Cross product	
$\sphericalangle(\mathbf{v}, \mathbf{u})$	`v.angle(u)`	Angle between two vectors	
$\hat{\mathbf{v}}$	`v.versor()`	Versor associated	$*$
$\hat{\mathbf{v}}$	`v.versorDl()`	Versor associated (dimensionless)	
$\pm\lvert\mathbf{v}\rvert/\lvert\mathbf{u}\rvert$ if $\mathbf{v} \parallel \mathbf{u}$	`v.scale(u)`	Ratio between parallel vectors	
	`v.rotate(a,ax)`	Rotate around z-axis ($2D$) or an arbitrary axis ($3D$)	$*$
$\mathbf{v} \oplus \mathbf{u}$	`v.directSum(u)`	Direct sum	
$\mathbf{v} \odot \mathbf{u}$	`v.elementwiseProduct()`	Element-wise product	
$\mathbf{v} \oslash \mathbf{u}$	`v.elementwiseDivision()`	Element-wise division	
$\lvert\mathbf{v}\rvert$	`v.norm()`	Euclidean norm	
$\lvert\mathbf{v}\rvert_1$	`v.normL1()`	L_1 norm	
$\lvert\mathbf{v}\rvert_p$	`v.normL(p)`	p norm	
$\lvert\mathbf{v}\rvert_\infty$	`v.normLInf()`	Infinity norm	
$\lvert\mathbf{v}\rvert^2$	`v.normSquared()`	Euclidean norm squared	
$\mathbf{v} \parallel \mathbf{u}$	`v.isParallel(u)`	Are parallel	
$\mathbf{v} \perp \mathbf{u}$	`v.isPerpendicular(u)`	Are perpendicular	
$\mathbf{v} \uparrow\uparrow \mathbf{u}$	`v.isSameDirection(u)`	Are parallel and have same directions	
$\mathbf{v} \uparrow\downarrow \mathbf{u}$	`v.isOppositeDirection(u)`	Are parallel and have opposite directions	
$\mathbf{v} = \mathbf{u}$	`v.isNear(u)`	Are the same	
$\mathbf{v} = 0$	`v.isNull()`	Is zero	
	`v.isNan()`	Has any NaN component	
	`v.isNormal()`	Are all components neither infinity, NaN, zero or subnormal	
$\mathbf{v} \neq \pm\infty$	`v.isFinite()`	Are all components finite	
$\mathbf{v} = \pm\infty$	`v.isInfinite()`	Has any infinite component	

Table 1. List of functions implemented for each type of vector. Cross product is only implemented for $3D$ vectors, while the angle is implemented only for $2D$ and $3D$ vectors. The $*$ marks the disponibility of an extra operation, which starts with `do` prefix (e.g. `doVersor`), that works inplace on the vector.

Operation	Method	Description	Note
$\mathbf{q}^*$	`q.conj()`	Conjugate	$*$
$\mathbf{q}^{-1}$	`q.inv()`	Reciprocal	
$0 + b\mathbf{i} + c\mathbf{j} + d\mathbf{k} + \ldots$	`q.pure()`	Vector complex, quaternion, octonion, etc.	
$a + 0\mathbf{i} + 0\mathbf{j} + 0\mathbf{k} + \ldots$	`q.scalar()`	Scalar complex, quaternion, octonion, etc.	
$(b, c, d, \ldots)$	`q.vector()`	Associated $n-1$ vector	
$2\arctan(q/a)$	`q.angle()`	Rotation angle	†
$(b, c, d)/\sin(\theta/2)$	`q.axis()`	Rotation axis	†
$\mathbf{qvq}^{-1}$	`q.rotate(v)`	Rotation of a vector	
$\mathcal{M}$	`q.matrix<n>()`	Matrix representation	
$a = 0$	`q.isPure()`	Real part is zero	
$(b, c, d, \ldots) = 0$	`q.isScalar()`	Unreal part is zero	
$\mathbf{q} = \mathbf{p}$	`q.isNear(p)`	Are the same	
$\mathbf{q} = 0$	`q.isNull()`	Is zero	
	`q.isNan()`	Has any NaN component	
	`q.isNormal()`	All components are neither infinity, NaN, zero or subnormal	
$\mathbf{q} \neq \pm\infty$	`q.isFinite()`	All components are finite	
$\mathbf{q} = \pm\infty$	`q.isInfinite()`	Has any infinite component	

Table 2. List of functions implemented for complex, quaternions, octonions, and sedenions, in addition to the one presented in Table 1. Methods marked with † are applicable to quaternions only. The $*$ marks the availability of an extra operation, which starts with `do` prefix (e.g. `doConj`), that works inplace on the quantity.

Operator	Operation	Note
`u + v`	$(u_1 + v_1, u_2 + v_2, \ldots, u_n + v_n)$	
`u - v`	$(u_1 - v_1, u_2 - v_2, \ldots, u_n - v_n)$	
`u * v`	*(Hamilton multiplication)*	†
`v * k`	$(v_1\,k, v_2\,k, \ldots, v_n\,k)$	
`k * v`	$(k\,v_1, k\,v_2, \ldots, k\,v_n)$	
`u / v`	*(Hamilton division)*	†
`v / k`	$(v_1/k, v_2/k, \ldots, v_n/k)$	
`u > v`	$u_1^2 + u_2^2 + \ldots + u_n^2 > v_1^2 + v_2^2 + \ldots + v_n^2$	
`u < v`	$u_1^2 + u_2^2 + \ldots + u_n^2 < v_1^2 + v_2^2 + \ldots + v_n^2$	
`u >= v`	$u_1^2 + u_2^2 + \ldots + u_n^2 \geq v_1^2 + v_2^2 + \ldots + v_n^2$	
`u <= v`	$u_1^2 + u_2^2 + \ldots + u_n^2 \leq v_1^2 + v_2^2 + \ldots + v_n^2$	
`u == v`	$u_1 = v_1$ and $u_2 = v_2$ … and $u_n = v_n$	
`u != v`	$u_1 \neq v_1$ or $u_2 \neq v_2$ … or $u_n \neq v_n$	

Table 3. List of operators implemented for each type of n-dimensional vector, quaternions, octonions, sedenions, and complex numbers. The † symbol marks hamiltonian multiplication and division, operations available only for quaternions, octonions, sedenions, and complex numbers. Relational operators compare the module of the vectors at the first instance: if the module is equal, then the vector's elements are compared one by one to obtain a strong ordering.

Class name	Description
`TranslateFrame`	Linear translation by a compatible vector
`RotateFrame2D`	2*D* rotation (by angle or complex)
`RotateFrame3D`	3*D* rotation (by Euler angles or quaternion)
`ScaleFrame`	Scaling by a scalar or by a different value for each axis
`CompositeFrame`	Wrap two transformation in a single one

Table 4. List of ready to use reference frame transformations, that can be used to describe complex frame transformations. The class `CompositeFrame` is rarely used directly but can be created combining the other transformations using the >> operator.

same classes for these entities, marking complex quantities as vectors with a negative dimensional number. In this way, we can represent complex numbers as `VectorC<-2>`, quaternions as `VectorC<-4>`, etc., enabling and disabling some methods using the N template parameter. However, for better readability, a set of alias for common entities is provided, as shown in Table 5. This solution allowed a high degree of code reuse and painless conversion between the entities without requiring inheritance and virtual method resolutions at runtime.

Matrix operations

During the development of the vectorial operations, we realized that matrix operations would be needed soon, which shall also benefit from the same UoM validation. The main difficulty was that not all operations were available for any matrices. For example, an operation like the determinant of a matrix is defined only for a square matrix. For this reason, we made heavy use of the `C++` concept `requires`. Although this severely limits its usage only within projects that use at least `C++20`, it allows a clear definition of the function usage constraints, and produces meaningful error messages when these conditions are not met.

As a design decision, matrix dimensions can be set using integer templates, as shown in Table 5: this means that matrix dimension can not be changed at runtime, allowing non-negligible performance optimizations by the compiler. As we can observe in Tables 6, 7 and 8, we decided to limit the method's implementations to the one which have a deterministic and non-iterative solution, omitting methods requiring matrices decomposition. In a future release, we may add support for this class of methods, allowing the calculation of eigenvalues, eigenvectors, matrix L_2 norms and similar quantities.

Floating point limitations and tolerances

Representation of floating-point number is a tricky point in any software package that handle non-integer quantities. The IEEE Standard for Floating-Point Arithmetic (`IEEE 754`)[14], is the de facto standard in modern computers, and `C++` supports it using the `float` and `double` variable types. This format allows storing any number inside a fixed size mantissa and exponent, allowing the representation of up to 1×10^{308} or down to 1×10^{-308}, with around 15-digits precision.

One first issue is that some rational numbers that are perfectly representable on a base-10 numeric system can not be expressed on a base-2 numeric system and inevitably introduce a rounding error.

Another issue is that trigonometric and transcendental functions in `C++` are calculated using a series sum and are truncated after a specific precision is achieved. This is usually not a problem in the engineering and scientific

Name	Description	Alias of
`VectorC<N>`	Generic *n*-dimensional Cartesian vector	
`VectorC1D`	Alias for 1*D* vectors	`VectorC<1>`
`VectorC2D`	Alias for 2*D* vectors	`VectorC<2>`
`VectorC3D`	Alias for 3*D* vectors	`VectorC<3>`
`VectorP2D`	Vector 2*D* in polar coordinates	
`VectorS3D`	Vector 3*D* in spherical coordinates	
`VectorY3D`	Vector 3*D* in cylindrical coordinates	
`Real`	Real quantity	`VectorC<-1>`
`Complex`	Complex quantity	`VectorC<-2>`
`Quaternion`	Quaternion quantity	`VectorC<-4>`
`Octonion`	Octonion quantity	`VectorC<-8>`
`Sedenion`	Sedenion quantity	`VectorC<-16>`
`Matrix<M,N>`	$M \times N$ generic matrix	
`RMatrix2`	2×2 square real-matrix	`Matrix<2,2,Real>`
`RMatrix3`	3×3 square real-matrix	`Matrix<3,3,Real>`
`RMatrix4`	4×4 square real-matrix	`Matrix<4,4,Real>`
`CMatrix2`	2×2 square complex-matrix	`Matrix<2,2,Complex>`
`CMatrix3`	3×3 square complex-matrix	`Matrix<3,3,Complex>`
`CMatrix4`	4×4 square complex-matrix	`Matrix<4,4,Complex>`
`HMatrix2`	2×2 square quaternion-matrix	`Matrix<2,2,Quaternion>`
`HMatrix3`	3×3 square quaternion-matrix	`Matrix<3,3,Quaternion>`
`HMatrix4`	4×4 square quaternion-matrix	`Matrix 4,4,Quaternion>`
`OMatrix2`	2×2 square octonion-matrix	`Matrix<2,2,Octonion>`
`OMatrix3`	3×3 square octonion-matrix	`Matrix<3,3,Octonion>`
`OMatrix4`	4×4 square octonion-matrix	`Matrix<4,4,Octonion>`
`SMatrix2`	2×2 square sedenion-matrix	`Matrix<2,2,Sedenion>`
`SMatrix3`	3×3 square sedenion-matrix	`Matrix<3,3,Sedenion>`
`SMatrix4`	4×4 square sedenion-matrix	`Matrix<4,4,Sedenion>`

Table 5. List of classes and aliases implemented in `Univec`.

fields because we are interested in the approximate values of our calculation, given that we do not fall into known floating-point pitfalls. However, our library is quite sensitive about this issue because often, we have to check if two vectors are parallel or perpendicular, or if a matrix is diagonal: these actions do require to check the result against the exact values, which is tricky when working with floating-point quantities.

To manage this last issue, we introduced the concept of tolerance, represented by the class `Tolerance`. This class can be passed as a template parameter to any function that requires comparing two values for equality (usually, but not always, the methods starting with the "is" prefix). We provide a default implementation which compares two values as equals if they equal to the 10th decimal digit. While this approach is not quite robust, it has the advantage of being relatively fast and easy to understand. If a different approach is required, the user can provide a custom class for all the methods or just for some specific calls.

Frame transformations

While we were using `Univec` we found that some formulas become much simpler when we use specific coordinates transformation. One good example comes from the simulation of microscopic electron-molecule non-relativistic interactions in the center-of-momentum reference frame. A transformation-matrix can represent any linear transformation: complex transformations can be easily represented and computed by chaining multiple simpler transformations such as translations, rotations, and scaling operations.

One can make a frame transformation using a concrete implementation of the abstract class `BaseFrameC3D` (in Table 4, we present ready-to-use implementations).

A vector can be easily transformed into a new reference frame:

```
VectorC3D<QtySiLength>         translation(0. * meter, 1. * meter, 0. * meter);
TranslateFrame<3,QtySiLength> frame(translation);
VectorC3D<QtySiLength>         globalPosition;                                  // ( 0,  0,  0) m
VectorC3D<QtySiLength>         localPosition = frame.forward(globalPosition); // ( 0, -1,  0) m
localPosition += VectorC3D<QtySiLength>(1. * meter, 1. * meter, 1. * meter); // ( 1,  0,  1) m
globalPosition = frame.backward(localPosition);                               // ( 1,  1,  1) m
```

Operation	Method	Description	Note
$\mathcal{A}_{i-1,j-1}$	`a(i,j)`	Element access (1-based)	
$\mathcal{A}_{i,j}$	`a[i,j]`	Element access (0-based)	
	`a.rowMajor(x)`	Element access (row-major)	
	`a.colMajor(x)`	Element access (column-major)	
m	`a.rows`	Rows count	
n	`a.columns`	Columns count	
$\mathcal{A}_{i,*}$	`a.row()`	Returns a specified row	
$\mathcal{A}_{*,j}$	`a.col()`	Returns a specified column	
$m = n$	`a.square`	Check if square	
$\mathcal{A}_{i_2,*} \leftarrow \mathcal{A}_{i_2,*} + \mathcal{A}_{i_1,*}$	`a.rowAdd(i1, i2)`	Row addition and store	$*$
$\mathcal{A}_{i_2,*} \leftarrow \mathcal{A}_{i_2,*} - \mathcal{A}_{i_1,*}$	`a.rowSub(i1, i2)`	Row subtraction and store	$*$
$\mathcal{A}_{i,*} \leftarrow \mathcal{A}_{i,*} \cdot s$	`a.rowMul(i, s)`	Row multiplication	$*$
$\mathcal{A}_{i,*} \leftarrow \mathcal{A}_{i,*}/s$	`a.rowDiv(i, s)`	Row division	$*$
$\mathcal{A}_{i_1,*} \leftrightarrow \mathcal{A}_{i_2,*}$	`a.rowSwap(i1, i2)`	Row swap	$*$
	`a.reverseRows()`	Reverse the rows	$*$
	`a.shuffleRows()`	Shuffles the rows	$*$
	`a.sortRows()`	Sorts the rows	$*$
$\mathcal{A}_{*,j_2} \leftarrow \mathcal{A}_{*,j_2} + \mathcal{A}_{*,j_1}$	`a.colAdd(j1, j2)`	Column addition and store	$*$
$\mathcal{A}_{*,j_2} \leftarrow \mathcal{A}_{*,j_2} - \mathcal{A}_{*,j_1}$	`a.colSub(j1, j2)`	Column subtraction and store	$*$
$\mathcal{A}_{*,j} \leftarrow \mathcal{A}_{*,j} \cdot s$	`a.colMul(j, s)`	Column multiplication	$*$
$\mathcal{A}_{*,j} \leftarrow \mathcal{A}_{*,j}/s$	`a.colDiv(j, s)`	Column division	$*$
$\mathcal{A}_{*,j_1} \leftrightarrow \mathcal{A}_{*,j_2}$	`a.colSwap(j1, j2)`	Column swap	$*$
	`a.reverseCols()`	Reverse the columns	$*$
	`a.shuffleCols()`	Shuffle the columns	$*$
	`a.sortCols()`	Sorts the columns	$*$
$o(\mathcal{A})$	`a.order()`	Order	†
$O(\mathcal{A})$	`a.orders()`	Order (in $m \times n$ form)	
$\text{adj}(\mathcal{A})$	`a.adj()`	Ajugate	†
$c_{m,n}$	`a.cofactor(m,n)`	Cofactor element	†
$C_{m,n}$	`a.cofactors(m,n)`	Cofactor matrix	†
$\det(\mathcal{A})$	`a.det()`	Determinant	†
$\text{tr}(\mathcal{A})$	`a.tr()`	Trace	†
$\mathcal{A}^T$	`a.t()`	Transpose	$*$
$\overline{A}$	`a.conj()`	Conjugate	$*$
$\mathcal{A}^H$	`a.conjT()`	Conjugate transpose	$*$
$\mathcal{A}_{i,j} \leftarrow 0 \quad \text{if} \lvert A_{i,j} \rvert < t$	`a.zap(t)`	Set to zero small elements	$*$
$\mathcal{A}^{-1}$	`a.inv()`	Inverse	†
$\mathcal{A}^+$	`a.pseudoInv()`	Moore–Penrose inverse	
$\mathcal{A}[m; n]$	`a.sub(n,m)`	Submatrix	
$m_{m,n}$	`a.minor(m,n)`	Minor element	†
$M_{m,n}$	`a.minors(m,n)`	Minor matrix	†
$(\mathcal{A}_{1,1}, \mathcal{A}_{2,2}, \ldots, \mathcal{A}_{n,n})$	`a.diagonal()`	Diagonal elements	†
$\mathcal{A}_{1,1} \cdot \mathcal{A}_{2,2} \cdot \ldots \cdot \mathcal{A}_{n,n})$	`a.diagonalProduct()`	Product of the diagonal elements	†
$\langle \mathcal{A}, \mathcal{B} \rangle_F$	`a.frobenius(b)`	Frobenius inner product	
$\sum \mathcal{A}_{i,j}$	`a.grandSum()`	Sum of all elements	
$\mathcal{A} \oplus \mathcal{B}$	`a.directSum(b)`	Direct sum	
$\mathcal{A} \otimes \mathcal{I}_b + \mathcal{I}_a \otimes \mathcal{B}$	`a.kroneckerSum(b)`	Kronecker sum	
$\mathcal{A} \otimes \mathcal{B}$	`a.kroneckerProduct(b)`	Kronecker product	
$\mathcal{A}_{i,j} \cdot \mathcal{B}_{i,j}$	`a.elementwiseProduct()`	Element-wise product	
$\mathcal{A}_{i,j}/\mathcal{B}_{i,j}$	`a.elementwiseDivision()`	Element-wise divisison	
	`a.reduction(type)`	Performs the reduction of the matrix	$*$

Table 6. List of functions implemented for the `Matrix<M,N>` class. The † symbol marks operations available for square matrix only, while ‡ symbols is available for row or column matrix only. The $*$ marks the disponibility of an extra operation, which starts with `do` prefix (e.g. `doColSwap`), that works inplace on the matrix. Matrix indices are zero-based.

Operation	Method	Description	
$\mathcal{A}_{m,n} < \infty$	`a.isFinite()`	All elements are finite	
$\mathcal{A}_{m,n} = \pm\infty$	`a.isInfinite()`	Any element is not finite	
$\mathcal{A}_{m,n} \notin \mathbb{R}$	`a.isNan()`	Any element is a `nan`	
$\mathcal{A} \approx \mathcal{B}$	`a.isNear(b)`	Is approximate equals	
$\mathcal{A}_{m,n} \in \mathbb{R}\backslash\{0\}$	`a.isNormal()`	All elements are finite, real and non-zero	
$\mathcal{A}_{m,n} = 0$	`a.isNull()`	All elements are zero	
$\mathcal{A} = \mathcal{I}$	`a.isIdentity()`	Is identity	†
$\mathcal{A}^H = \mathcal{A}^{-1}$	`a.isUnitary()`	Is unitary	†
$\mathcal{A}_{m,n} = \overline{\mathcal{A}}_{n,m}$	`a.isHermitian()`	Is hermitian	†
$\mathcal{A}_{m\neq n} = 0$	`a.isDiagonal()`	Is diagonal	†
$\mathcal{A}\mathcal{A}^T = \mathcal{I}$	`a.isOrthogonal()`	Is orthogonal	†
$\det(\mathcal{A}) = 0$	`a.isSingular()`	Is singular	†
$\mathcal{A}_{m,n} = \mathcal{A}_{n,m}$	`a.isSymmetric()`	Is symmetric	†
$\mathcal{A}_{m,n} = -\overline{\mathcal{A}}_{n,m}$	`a.isSkewHermitian()`	Is skew-hermitian	†
$\mathcal{A}_{m,n} = -\mathcal{A}_{n,m}$	`a.isSkewSymmetric()`	Is skew-symmetric	†
$\mathcal{A}_{m,n} = -\mathcal{A}_{n,m}$	`a.isTriangular()`	Is triangular	†
$\mathcal{A}_{m<n} = 0$	`a.isLowerTriangular()`	Is lower-triangular	†
$\mathcal{A}_{m>n} = 0$	`a.isUpperTriangular()`	Is upper-triangular	†
	`a.isStrictTriangular()`	Is strict-triangular	†
	`a.isRowEchelon()`	Is row echelon form	
	`a.isColEchelon()`	Is column echelon form	
	`a.isRedRowEchelon()`	Is reduced row echelon form	
	`a.isRedColEchelon()`	Is reduced column echelon form	
$\|\mathcal{A}\|$	`a.norm()`	Euclidean row or column norm	‡
$\|\mathcal{A}\|^2$	`a.normSquared()`	Euclidean row or column norm squared	‡
$\|\mathcal{A}\|_1$	`a.normL1()`	L_1 norm	
$\|\mathcal{A}\|_{max}$	`a.normLMax()`	Max norm	
$\|\mathcal{A}\|_\infty$	`a.normLInf()`	Infinity norm	
$\|\mathcal{A}\|_{p,q}$	`a.normL(p,q)`	pq norm	
$\|\mathcal{A}\|_F$	`a.normF()`	Frobenius norm	

Table 7. List of functions implemented for the `Matrix<M,N>` class. The † symbol marks operations available for square matrix only, while ‡ symbols is available for row or column matrix only.

Operation	Operator	Description
$\mathcal{A} + \mathcal{B}$	`a + b`	Sum
$\mathcal{A} - \mathcal{B}$	`a - b`	Subtraction
$\mathcal{A}\,\mathcal{B}$	`a * b`	Multiplication
$\mathcal{A}\,c$	`a * c`	Multiplication with a scalar
$c\,\mathcal{A}$	`c * a`	Multiplication with a scalar
$\mathcal{A}\,\mathbf{v}$	`a * v`	Multiplication with a vector
$\mathcal{A}/c$	`a / c`	Division by a scalar

Table 8. List of functions implemented for the `Matrix<M,N>` class.

Multiple transformations can be combined to describe complex scenarios using the syntax:

```
VectorC3D<QtySiLength> v1; // (0,0,0) m
VectorC3D<QtySiLength> v2(0. * meter, -sqrt(2) * meter, sqrt(2) * meter);

VectorC3D<QtySiLength>       translation(0. * meter, 1. * meter, 0. * meter);
EulerRotation3D<QtySiLength> rotation(
        45. * degrees, 0. * degrees, 0. * degrees,
        RotationMode::INTRINSIC,
        RotationAxis::X, RotationAxis::Y, RotationAxis::Z);
QtySiDimensionless           scale(0.5);

auto frame = TranslateFrame<3,QtySiLength>(translation)
        >> RotateFrame3D<QtySiLength>(rotation)
        >> ScaleFrame<3,QtySiLength>(scale);

assert(frame.forward(v1).isNear(v2));
assert(frame.backward(v2).isNear(v1));
```

Vector conversion

Another valuable feature of `Univec` is changing the coordinate system of a vector. The conversion from a type to another is made via a constructor, which is explicit when the conversion is computationally expensive (e.g. `VectorP2D` to `VectorC<2>`), and implicit otherwise (e.g. `VectorC<4>` to `Quaternion`).

These methods can be useful for executing operations on vectors represented in different coordinate systems. For example, for converting a `VectorP2D` into a `VectorC2D`, we can use the following code:

```
VectorP2D<QtySiLength> vec_a(5. * meters,  90. * degrees);

// Cartesian coordinates
VectorC2D<QtySiLength> vec_b(vec_a); // ( 0, 5) m
```

while for 3*D* vectors, we can use:

```
VectorC3D<QtySiLength> vec_a(3. * meters, 4. * meters, 12. * meters);

// Spherical coordinates
VectorS3D<QtySiLength> vec_b(vec_a); // ( 13 m, 22 deg, 53 deg)

// Cylindrical coordinates
VectorY3D<QtySiLength> vec_c(vec_a); // ( 5 m, 53 deg, 12 m)
```

and for complex and quaternions, we have:

```
Complex<QtySiFrequency> c(1. * hertz,  2. * hertz);
Quaternion<QtySiLength> q(1. * meters, 2. * meters, 3. * meters, 4. * meters);

// 2x2 matrix
Matrix<2,2,QtySiFrequency> m1 = c.matrix();

// 4x4 matrix
Matrix<4,4,QtySiLength>    m2 = q.matrix();
```

Usage examples

In this subsection, we will present two realistic examples of using `Univec`. For the first example we will show a simple implementation of the Bethe–Bloch equation, a handy formula for evaluating the energy deposit of ionizing charged particles. For the second example we will show how our library can be used in elementary particle physics, evaluating the decay rate of the W^- boson. It is important to notice that we perform the calculation using SI units and not the natural units that are normally used. This is because there is currently no support for natural units in `Boost::Units`, but we plan to address this issue in the near future (see "Limitations and future plans"). All the examples presented in this article are available in a public repository[15].

Bethe–Bloch

The well-known Bethe–Bloch formula[16–18] is used to calculate the mean energy loss of a charged particle. For example, if we have a particle with velocity v, charge z (in a unit of elementary charges), traveling a medium

with electron number density n and mean excitation energy I, we can calculate the mean energy-loss using the formula:

$$-\left\langle \frac{dE}{dx} \right\rangle = \frac{4\pi}{m_e c^2} \cdot \frac{nz^2}{\beta^2} \cdot \left(\frac{e^2}{4\pi\varepsilon_0} \right)^2 \cdot \left[\ln\left(\frac{2m_e c^2 \beta^2 \gamma^2}{I} \right) - \beta^2 \right] \tag{1}$$

where m_e is the electron rest mass, c is the speed of light and $\beta = v/c$ as well as $\gamma = 1/\sqrt{1-\beta^2}$ are the Lorentz factors.

Using `Univec`, we can encode this formula into a compact expression, enjoying a fully dimensional analysis validation:

```
QtySiDimensionless molZ            = 10; // Neon
QtySiDimensionless eleChargeNumber = -1;
QtySiDimensionless pi (M_PI);
QtySiEnergy        eleRestEnergy (m_e * c * c);
QtySiEnergy        eleKinEnergy  (1000. * kilo * volt * e);
QtySiDensity       molDensity    (2.6867811e25 / cubic_meter);
QtySiDensity       eleDensity    (molZ * molDensity);
QtySiEnergy        meanExcEnergy (137. * volt * e);

QtySiDimensionless gamma  = (eleRestEnergy + eleKinEnergy) / eleRestEnergy;
QtySiDimensionless gamma2 = gamma * gamma;
QtySiDimensionless beta2  = 1. - 1. / gamma2;

auto               p1 = 4. * pi / (m_e * c * c);
QtySiDensity       p2 = eleDensity * eleChargeNumber * eleChargeNumber / beta2;
auto               p3 = e * e / (4. * pi * epsilon_0);
QtySiDimensionless p4 = log(2. * m_e * c * c * beta2 * gamma2 / meanExcEnergy) - beta2;

cout << "Mean energy loss: "
     << (double) ((-p1 * p2 * p3 * p3 * p4) / (1e3 * volt * e / meter)) << " keV/m" << endl;
```

$W^- \to e^- + \bar{\nu}_e$ decay

The previous example was helpful for presenting the dimensional analysis validation, but the equation itself only contained scalar operation. We will show here how vector, spinor, and matrix operations can be combined in a compact syntax to perform advanced calculations. In this example we reproduce a derivation from[19], ch. 15 which describes the properties of the W and Z bosons. To calculate the decay rate of a W^- boson into an electron, e^-, and an electronic anti-neutrino, $\overline{\nu}_e$ (shown in Fig. 1), we calculate the decay matrix elements using

$$\mathcal{M}_{fi} = \frac{g_W}{\sqrt{2}} \epsilon_\mu^\lambda(p_1)\bar{u}(p_3)\gamma^\mu \frac{1}{2}(1-\gamma^5)v(p_4) \tag{2}$$

where γ^μ are the γ-matrices in Dirac–Pauli representation and ϵ_μ^λ represents the three possible polarization states

$$\epsilon_-^\mu = \frac{1}{\sqrt{2}}(0,1,-i,0), \qquad \epsilon_L^\mu = \frac{1}{m_W}(p_z,0,0,E), \qquad \epsilon_+^\mu = -\frac{1}{\sqrt{2}}(0,1,i,0) \tag{3}$$

and $v(p_4), \bar{u}(p_3)$ are, respectively, the adjoint particle spinor and antiparticle spinor, defined as

$$u_1(p) = N\begin{pmatrix} 1 \\ 0 \\ \frac{p_z}{E+m} \\ \frac{p_x+ip_0}{E+m} \end{pmatrix}, \quad u_2(p) = N\begin{pmatrix} 0 \\ 1 \\ \frac{p_x-ip_0}{E+m} \\ \frac{-p_z}{E+m} \end{pmatrix}, \quad v_1(p) = N\begin{pmatrix} \frac{p_x-ip_0}{E+m} \\ \frac{-p_z}{E+m} \\ 0 \\ 1 \end{pmatrix}, \quad v_2(p) = N\begin{pmatrix} \frac{p_z}{E+m} \\ \frac{p_x+ip_0}{E+m} \\ 1 \\ 0 \end{pmatrix} \tag{4}$$

with $N = \sqrt{E+m}$. Finally, having $\mathcal{M}_{fi}$, we can calculate the average decay rate using

$$\left\langle |\mathcal{M}_{fi}|^2 \right\rangle = \frac{1}{3}\left(|\mathcal{M}_-|^2 + |\mathcal{M}_L|^2 + |\mathcal{M}_+|^2\right) \tag{5}$$

and

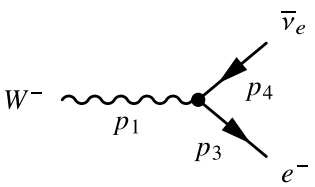


Figure 1. Lowest-order Feynman diagram for W^- boson decay into an electron and an anti-neutrino[19].

$$\Gamma = \frac{p^*}{8\pi m_W^2} \left\langle \left| \mathcal{M}_{fi} \right|^2 \right\rangle \tag{6}$$

where $p*$ is the momentum of the final particle in the center-of-mass frame.

The equations above are dimensional-invalid in SI units (e.g., we sum a mass and energy). However, in natural units, we define quantities like mass, momentum, length, etc., in equivalent energy units, so their operations are legit. Unfortunately, our current implementation does not support natural units, yet, so we are forced to convert to SI units to perform this calculation. The result is the code below, which uses our custom classes for matrices, vectors, and complex values to produce a compact and readable code with a full dimensional check at compile time:

```
QtySiEnergy eV = 1. * volt * e;
QtySiPlaneAngle pi = M_PI * radians;
auto h_bar = h / (2. * pi); // Energy * Time
QtySiDimensionless gW = e / (5.28e-19 * coulomb) / sqrt(0.231214);

QtySiEnergy P1_TotalEnergy (200.    * giga * eV);
QtySiEnergy P1_RestEnergy  ( 80.433 * giga * eV);
VectorC3D<> P1_Direction = VectorC3D<>::kZ();
// Using: E² = (pc)² + (m₀c²)²
VectorC3D<QtySiMomentum> P1_Momentum = sqrt(P1_TotalEnergy * P1_TotalEnergy - P1_RestEnergy *
↪  P1_RestEnergy) / c * P1_Direction;

QtySiPlaneAngle P3_Theta (45. * degrees);
VectorC3D<> P3_Direction = VectorC3D<>::kZ().rotateY(P3_Theta/2.);
VectorC3D<QtySiMomentum> P3_Momentum = P1_Momentum.norm() / 2. * P3_Direction;
QtySiEnergy P3_RestEnergy = m_e * c * c;
QtySiEnergy P3_TotalEnergy = sqrt(P3_RestEnergy * P3_RestEnergy + P3_Momentum.normSquared() * c * c);

QtySiPlaneAngle P4_Theta (-45. * degree::degrees);
VectorC3D<> P4_Direction = VectorC3D<>::kZ().rotateY(P4_Theta/2.);
VectorC3D<QtySiMomentum> P4_Momentum = P1_Momentum.norm() / 2. * P4_Direction;
QtySiEnergy P4_RestEnergy  (0.5 * eV);
QtySiEnergy P4_TotalEnergy = sqrt(P4_RestEnergy * P4_RestEnergy + P4_Momentum.normSquared() * c * c);

// P3 u1 spinor
auto P3_u1= sqrt(P3_TotalEnergy) * CMatrix<4,1> (
        Complex<>(1),
        Complex<>(0),
        Complex<QtySiEnergy>( P3_Momentum.z * c) / P3_TotalEnergy,
        Complex<QtySiEnergy>( P3_Momentum.x * c, P3_Momentum.y * c) / P3_TotalEnergy);

// P3 u2 spinor
auto P3_u2 = sqrt(P3_TotalEnergy) * CMatrix<4,1> (
        Complex<>(1),
        Complex<>(0),
        Complex<QtySiEnergy>( P3_Momentum.x * c, -P3_Momentum.y * c) / P3_TotalEnergy,
        Complex<QtySiEnergy>(-P3_Momentum.z * c) / P3_TotalEnergy);

// P4 v1 spinor
auto P4_v1 = sqrt(P4_TotalEnergy) * CMatrix<4,1> (
        Complex<QtySiEnergy>( P4_Momentum.x * c, -P4_Momentum.y * c) / P4_TotalEnergy,
        Complex<QtySiEnergy>(-P4_Momentum.z * c) / P4_TotalEnergy,
        Complex<>(0.),
        Complex<>(1.));

// P4 v2 spinor
auto P4_v2 = sqrt(P4_TotalEnergy) * CMatrix<4,1> (
        Complex<QtySiEnergy>( P4_Momentum.z * c) / P4_TotalEnergy,
        Complex<QtySiEnergy>( P4_Momentum.x * c, P4_Momentum.y * c) / P4_TotalEnergy,
        Complex<>(1.),
        Complex<>(0.));

auto fn_adjoint = [](const auto& spinor) { return spinor.conjT() * diracGamma<0>; };
auto fn_controvariant = [](const auto& tensor) { return CMetricTensorG<> * tensor; };

CMatrix<4,1> polMinus =  1./sqrt(2) * CMatrix<4,1>( Complex<>(), Complex<>(1), Complex<>(0,-1),
↪  Complex<>());
CMatrix<4,1> polPlus  = -1./sqrt(2) * CMatrix<4,1>( Complex<>(), Complex<>(1), Complex<>(0, 1),
↪  Complex<>());
CMatrix<4,1> polLong  = -1./P1_RestEnergy * CMatrix<4,1,QtySiEnergy>(
                              Complex<QtySiEnergy>(P1_Momentum.z * c),
                              Complex<QtySiEnergy>(),
                              Complex<QtySiEnergy>(),
                              Complex<QtySiEnergy>(QtySiEnergy(P1_TotalEnergy)));

std::map<int,CMatrix<4,1>> polarisations;
polarisations[-1] = polMinus;
polarisations[ 0] = polLong;
polarisations[+1] = polPlus;
```

```
typename power_typeof_helper<QtySiEnergy,static_rational<2>>::type  M1_avg, M2_avg;
for (int mu = 0; mu <= 3; mu++)
{
    for (int pol = -1; pol <= 1; pol++)
    {
        CMatrix<4,1,QtySiEnergy> M1 = gW / sqrt(2) * fn_controvariant(polarisations[pol]) *
                                      fn_adjoint(P3_u1) * diracGammaMu(mu) *
                                      0.5 * (CI<4> - diracGamma<5>) * P4_v1;

        CMatrix<4,1,QtySiEnergy> M2 = gW / sqrt(2) * fn_controvariant(polarisations[pol]) *
                                      fn_adjoint(P3_u2) * diracGammaMu(mu) *
                                      0.5 * (CI<4> - diracGamma<5>) * P4_v2;

        M1_avg += M1.normSquared() / 3.;
        M2_avg += M2.normSquared() / 3.;
    }
}

QtySiFrequency gamma1 =
        P3_Momentum.norm() * c / (8. * pi * P1_RestEnergy * P1_RestEnergy * h_bar) * M1_avg;
QtySiFrequency gamma2 =
        P3_Momentum.norm() * c / (8. * pi * P1_RestEnergy * P1_RestEnergy * h_bar) * M2_avg;

cout << "  Γ1: " << gamma1 << "  τ1: " << 1./gamma1 << endl;
cout << "  Γ2: " << gamma2 << "  τ2: " << 1./gamma2 << endl;
```

As we have already explained, using SI units is not a natural choice for HEP physicists. Due to the nature of the subject, we have to handle SI units values that differ from each other by many orders of magnitude. Due to this fact, a substantial precision loss can occur during calculations (see "Limitations and future plans" for details). However, our example presented above produced pretty good results, near to the accepted value of $\tau \approx O(10^{-25}s)$.

Methods

An effective software library needs an efficient and reliable procedure for validating both the functionality, the correctness, and the performances obtained. Therefore, in implementing this library, we perform two type of tests: a test of the functionality and correctness of the implemented methods and a test of performance compared with similar techniques for calculating the same operation.

When we talk about the functionality and performance tests, even though the source code of this library was carefully written, we felt the need to add an additional layer of verification by integrating a testing system.

To deliver a reliable application, we have defined a good software development practice by incorporating a Test-Driven Development (TDD) system. More details about this integration are presented in "Unit testing". Additionally, we chose to use two different compilers, `GCC`[20] and `Clang`[21], to support the wide range of machines our software could run on. The amount of code executed during our tests is greater than 95% (according to `gcovr`[22]—the code coverage utility we use), ensuring good code coverage.

Unit testing

`Univec` was developed using the Test-Driven Development (TDD) software design paradigm where code development and testing occur simultaneously. By integrating a unit testing strategy, we can separately test for the correctness of our application's functionality. To implement these tests, `Univec` uses `GoogleTest`[23], one of the most popular `C++` unit testing frameworks. For each implemented method and operator of our code an associated test ensures that the expected behavior satisfies our specifications and that functionality is preserved.

We have chosen to integrate CI/CD, another modern software development practice, through automated integration of testing and code documentation into the stages of the software development. We use `GitLab`[24] to host our repository, allowing us to access the provided CI/CD feature. For each commit or push GitLab triggers the build of code, runs all unit tests implemented with `GoogleTest` and extracts the annotated `C++` files to generate the online `Univec` documentation using `Doxygen`[25]. Furthermore, at the end of each test procedure on the pipeline, several report files are generated: two files with the test results (one for each compiler: `junit_gcc.xml` and `junit_clang.xml`) and one file that briefly presents code coverage results for the tests run (`coverage_gcc.xml`).

This combined approach, CI/CD and unit tests, assures us of the `Univec`'s functionality and maintainability.

Performance study

Performance comparison against existing solutions is a crucial aspect in the introduction of a new library. We compare `Univec` methods against four different approaches of performing the same operations:

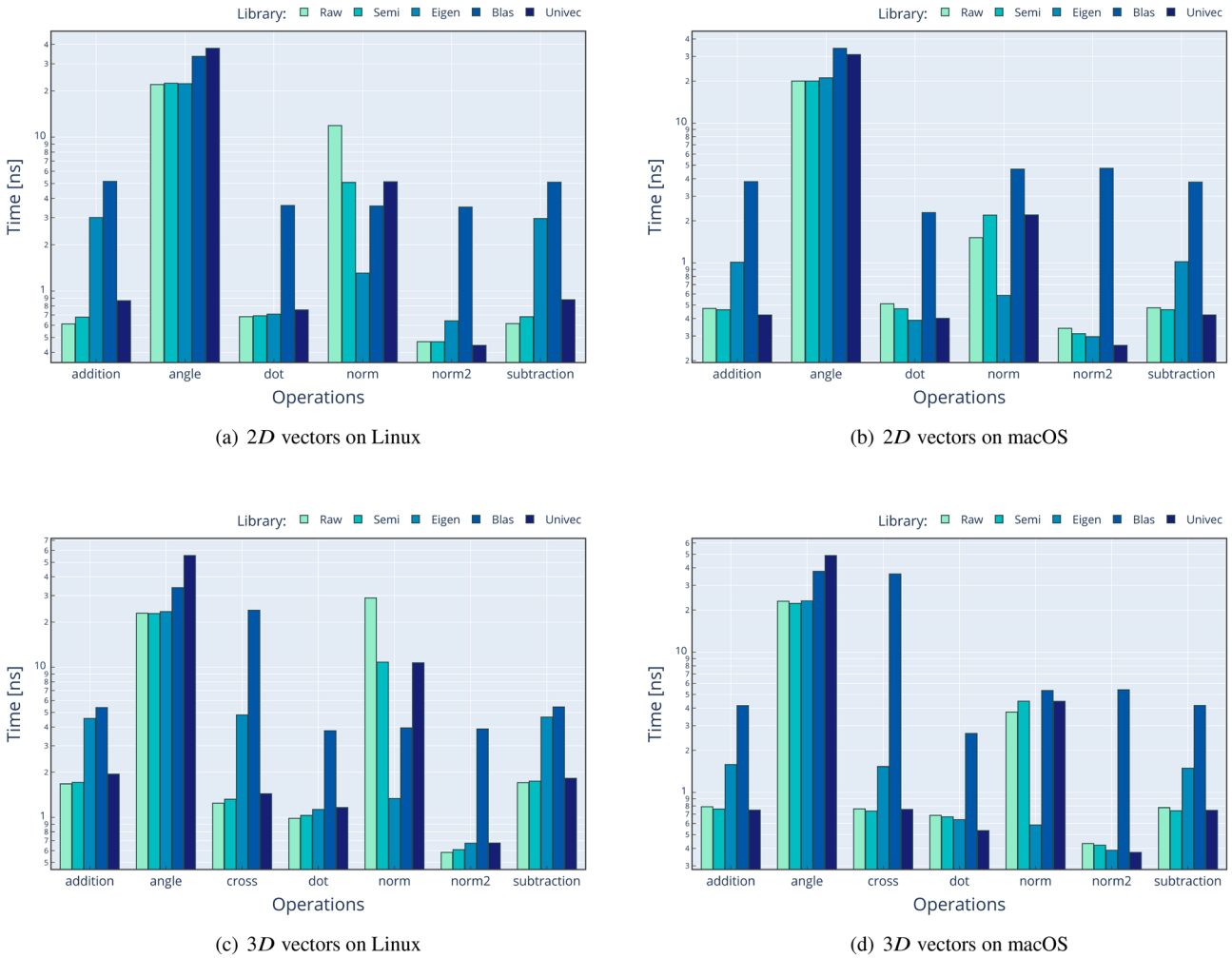
- **Raw**: using only primitive C++ types such as `double`.
- **Semi**: using `Boost::Units`[6] types and methods.
- **Eigen**[10]: using `Eigen v3`[10] types and methods.
- **BLAS**[26]**/LAPACK**[27]: using `OpenBLAS`[28] and `LAPACKE`[29] (for Linux) or the Accelerate framework[30] (for macOS) methods over primitive C++ types such as `double`.

When the operation is too complex to implement (e.g. determinant of $N \times N$ matrix, where $N>4$ for Raw and Semi approaches) or unavailable in a given library (e.g. cofactors of a 3×3 matrix for BLAS approach), the result is omitted. We compare a subset of operations, as shown in Figs. 2 and 3. The procedure of performance testing is performed using this scheme:

1. Input data are generated for Raw operation, using $n \cdot 10000$ random values between -1000 and 1000, with n being the number of operands, and then stored in contiguous memory locations.
2. Input data from Raw are copied to Semi, Eigen, Blas, and Univec. We use meter [m] unit for solutions with UoM validation.
3. The timer is started.
4. The operation is performed 10,000 times, without multi-threading, and results are stored in a contiguous memory location.
5. The timer is stopped, and the time spent for a single operation is calculated dividing the total time by 10,000.
6. The results for each approach are compared for correctness against the result of Raw implementation.
7. All these steps are repeated 1000 times in order to calculate the mean value and the stdev.

The source code for the performance analysis is available at the following locations:
https://gitlab.com/edystan/univec-performance-test
https://gitlab.com/edystan/univec-scaling-test

The first repository contains the performance tests for commonly used methods for both `Matrix` and `VectorC` classes, results being displayed in Figs. 2 and 3. In contrast, the second one contains the performance analysis for only two methods (vector dot product and matrix determinant) but for vector dimensions between $2D$ and $10D$ and matrices sizes between 2×2 and 10×10. For each of the two performance projects, the test



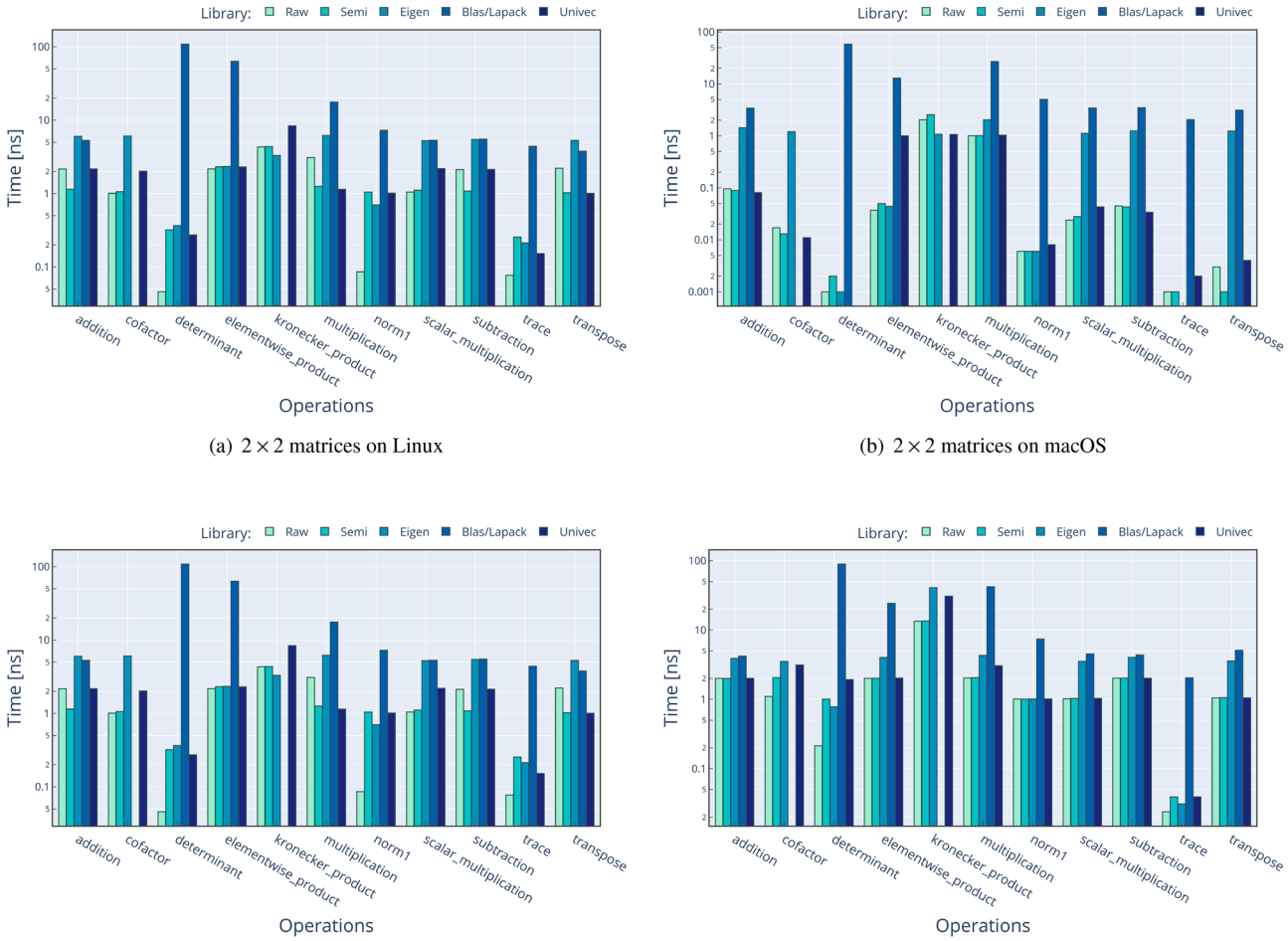
(a) $2D$ vectors on Linux

(b) $2D$ vectors on macOS

(c) $3D$ vectors on Linux

(d) $3D$ vectors on macOS

Figure 2. Time needed to perform the specified vector operations, using five different code implementations. The left side refers to Linux setup, while the right side refer to macOS setup. In this plot, lower values mean better performance.



(a) 2×2 matrices on Linux

(b) 2×2 matrices on macOS

(c) 3×3 matrices on Linux

(d) 3×3 matrices on macOS

Figure 3. Time needed to perform the specified matrix operations, using five different code implementations. The left side refers to Linux setup, while the right side refer to macOS setup. In this plot, lower values mean better performance.

results are collected and saved in `.csv` files, one for each operating system, and available at the previously mentioned addresses. As well, results for these tests are shown in Figs. 4 and 5.

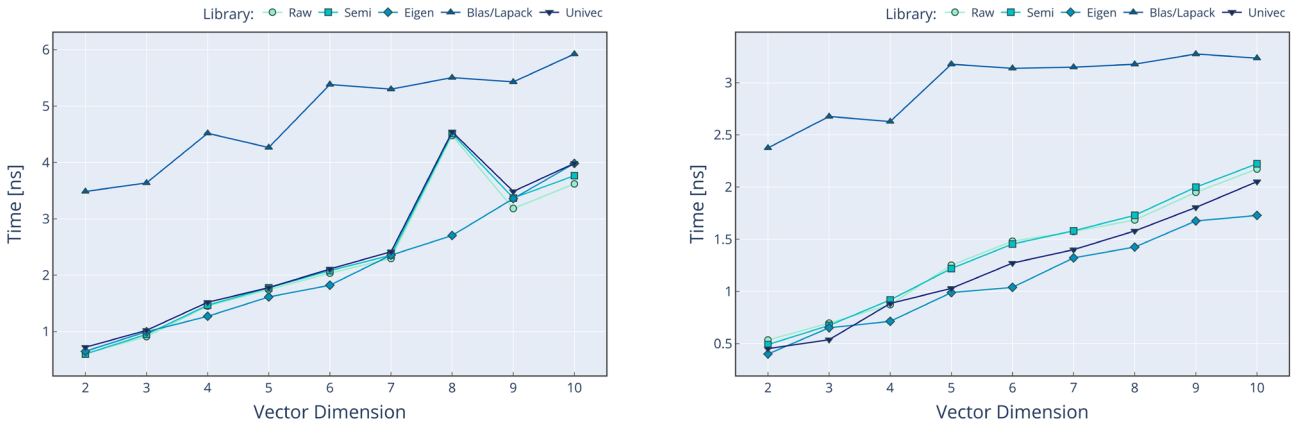
The performance validation was performed on two laptops to simulate real scenarios in two operating systems. The configurations are the following:

1. Lenovo V17 G2 ITL laptop with Intel Core i7-1165G7 at 2.80 GHz, 16 GB RAM. Ubuntu 22.04.2 LTS, Boost 1.74.0, Eigen 3.4.0, Univec 1.2, OpenBlas 3.10.0, Lapack 3.10.0. Compiled with GCC 11.2.0 with Release configuration.
2. Apple MacBook Pro laptop with Apple M2 Max CPU at max. 3.49 GHz, 32 GB RAM. macOS Ventura 13.4.1, Boost 1.82.0, Eigen 3.4.0, Univec 1.2, Accelerate-1 (for Blas/Lapack). Compiled with Apple Clang version 14.0.3 with Release configuration.

We can conclude that `Univec` performs in line with the other solutions, without any significant overhead, but with the addition of compile-time validation and a reasonable interface. In the performance analysis, we can see that Blas often shows poor performance, but we assume two factors cause this:

1. The overhead of calling a Fortran function from C++ code (when applicable).
2. The fact that we perform a BLAS call for each vector: we realize that BLAS can be used for some operations, but not all, in a SIMD (single instruction multiple data) fashion, which could mitigate the function calling overhead.

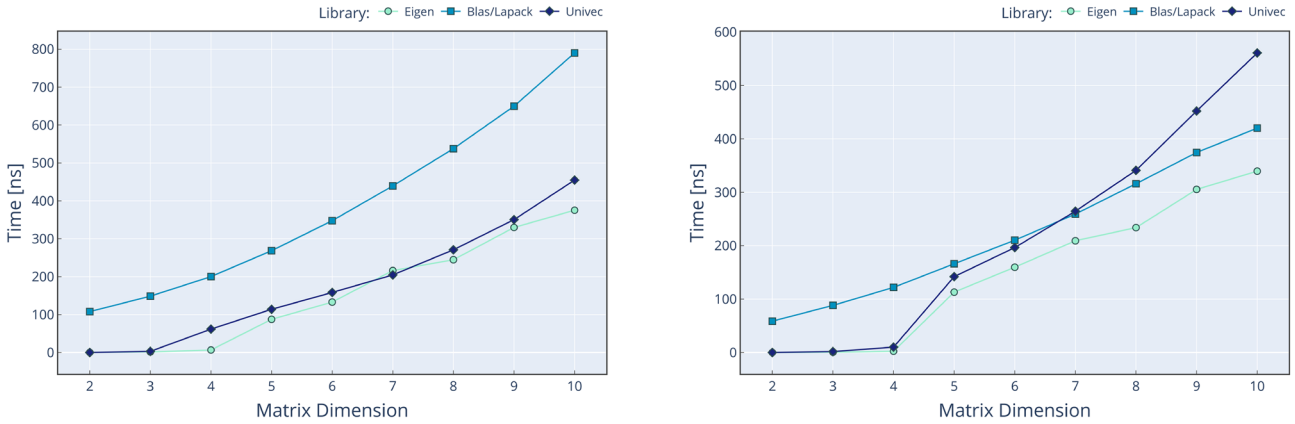
In addition to this, we noticed an anomaly on the calculation of dot product for 8*D* vectors: as shown in Fig. 4a, we can see that we have a performance hit on Linux/GCC platform, which seems to only appear at this specific dimension. We do not have a definitive explanation for this phenomenon, however we suspect that is a bug on



(a) Vector dot product on Linux

(b) Vector dot product on macOS

Figure 4. Time needed to perform the vector dot product for different vector sizes. The left side refers to Linux setup, while the right side refer to macOS setup. In this plot, lower values mean better performance.



(a) Matrix determinant on Linux

(b) Matrix determinant on macOS

Figure 5. Time needed to perform the matrix determinant for different matrix sizes. The left side refers to Linux setup, while the right side refer to macOS setup. In this plot, lower values mean better performance.

the GCC optimization algorithm, because the same issue is not present in macOS/Clang environment, as show in Fig. 4b.

Another anomaly we found in Fig. 5b, is that, for macOS, the timing of determinant do not scale well with the increasing of dimensionality. We believe that the causes of this behavior are:

1. The use the Gaussian elimination method for matrix bigger than 4×4, which is not the most efficient method available (BLAS uses the more efficient *PLU* decomposition).
2. Some different optimization settings applied by the Apple Clang compiler.

We plan to analyze and fix these optimization issues in the near future, in order to have performance in line with other implementations.

Limitations and future plans

Our strongest limitation is that, currently, our solution only works with `Clang` of version higher or equal to `14`. This is caused by our decision to use the latest `C++20` features like `requires` and `constexpr/consteval`. Compatibility with `GCC` is limited at this moment because we use the `__declspec(property)` attribute to give access to the vector components. `Univec` was not tested under the `MSVC` or `icc` compiler, however we do not expect blocking issues as long as `C++20` is supported.

The use of `Boost.Units` as an underlying library for `Univec` has several non-negligible weak points that, unfortunately, seem to have no solution in the near future. The first limitation is that this library is available only as part of a more comprehensive framework, `Boost`, which many software developers often see as

a heavy dependency. The second limitation is that the current implementation is based on heavy use of Boost Metaprogramming Library (MPL) and template meta-programming techniques, which produce error messages challenging to understand for the non-experienced software developers.

In "$W^- \to e^- + \bar{\nu}_e$ decay", we present an example of an elementary particle physics calculus implementation. However, the use of SI units is not encouraged and is not widespread in this field. We plan in a near future to add support to `Boost::Units`[6] for natural units, allowing more easy adoption of our solution by the HEP community.

There is a proposal to introduce the support for UoM-aware units directly in the standard language (see Mateusz Pusz's talk at CppCon 2019[31]). This approach will solve the problems presented above and will provide a more reliable solution with a less steep learning curve. We plan to create another implementation which uses a standard library implementation when it will become available. However, it will take several years until such implementation will find its way into the `C++` standard library. Last, but not least, we plan to introduce in the near future a SIMD optimization (e.g. `SSE 2` or the newer `AVX-512`), which can increase the performance by an order of magnitude when working with vectors or matrices.

Conclusions

In this paper, we presented `Univec`, our solution for UoM-validation in software development, aiming to improve the use of vector and matrix calculations in `C++` development. After a brief overview of the existing UoM solutions, we discussed their limitations and presented our solution, with several specific user-case scenarios.

The elaboration of `Univec` started as an internal project during the designing phase of our `Betaboltz`[32] project. While developing this project, we realized the benefits of integrating the UoM analysis directly into our source code, virtually removing the most common causes of mistakes. Even if this solution does not remove all the sources of error, `Univec` allowed us to focus on the main development workflow, increasing confidence in our implementation.

We plan to also create to a native implementation when a UoM-aware implementation will be published in the `C++` standard library and we hope that similar solutions will be widely accepted in the development of advanced computational calculus for the scientific community.

Data availability

All data generated or analysed during this study are included in this published article and its supplementary information files.

Received: 23 January 2023; Accepted: 27 September 2023
Published online: 04 October 2023

References

1. Mayerhofer, T., Wimmer, M. & Vallecillo, A. Adding uncertainty and units to quantity types in software models. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Software Language Engineering*, SLE 2016, 118–131. https://doi.org/10.1145/2997364.2997376 (Association for Computing Machinery, 2016).
2. McKeever, S., Bennich-Björkman, O. & Salah, O.-A. Unit of measurement libraries, their popularity and suitability. *Softw. Pract. Exp.* **51**, 711–734. https://doi.org/10.1002/spe.2926 (2021).
3. Preussner, G. M. Dimensional Analysis in Programming Languages. https://gmpreussner.com/research/dimensional-analysis-in-programming-languages (2018).
4. Apple Inc. Swift Language. https://swift.org (2022).
5. Kennedy, A. Types for Units-of-Measure: Theory and Practice. In *Central European Functional Programming School: Third Summer School, CEFP 2009, Budapest, Hungary, May 21–23, 2009 and Komárno, Slovakia, May 25–30, 2009, Revised Selected Lectures*, Lecture Notes in Computer Science (Horváth, Z., Plasmeijer, R. & Zsók, V., eds.), 268–305. https://doi.org/10.1007/978-3-642-17685-2_8 (Springer, 2010).
6. Schabel, M. C. & Watanabe, S. Chapter 44 boost.units 1.1.0-1.80.0. https://www.boost.org/doc/libs/1_80_0/doc/html/boost_units.html (2020).
7. Petty, G. W. Automated computation and consistency checking of physical dimensions and units in scientific programs. *Softw. Pract. Exp.* **31**, 1067–1076. https://doi.org/10.1002/spe.401 (2001).
8. Grecco, H. Pint. https://github.com/hgrecco/pint (2022).
9. Lönnblad, L. CLHEP-a project for designing a C++ class library for high energy physics. *Comput. Phys. Commun.* **84**, 307–316. https://doi.org/10.1016/0010-4655(94)90217-8 (1994).
10. Jacob, B. & Guennebaud, G. Eigen is a c++ template library for linear algebra: Matrices, vectors, numerical solvers, and related algorithms. https://eigen.tuxfamily.org/ (2010).
11. Dembinski, H., Nellen, L., Reininghaus, M. & Ulrich, R. Technical Foundations of CORSIKA 8: New Concepts for Scientific Computing. In *Proceedings of 36th International Cosmic Ray Conference—PoS(ICRC2019)*, vol. 358, 236. https://doi.org/10.22323/1.358.0236(2019).
12. Heck, D., Knapp, J., Capdevielle, J.-N., Schatz, G. & Thouw, T. *CORSIKA: A Monte Carlo Code to Simulate Extensive Air Showers* (Tech. Rep, Institut national de physique nucléaire et de physique des particules, 1998).
13. Moene, M. PhysUnits-CT-Cpp11. https://github.com/martinmoene/PhysUnits-CT-Cpp11 (2013).
14. IEEE Standard for Floating-Point Arithmetic. *IEEE Std 754-2019 (Revision of IEEE 754-2008)* 1–84. https://doi.org/10.1109/IEEESTD.2019.8766229(2019). Conference Name: IEEE Std 754-2019 (Revision of IEEE 754-2008).
15. Univec article examples -GitLab.
16. Bethe, H. Zür Theorie des Durchgangs schneller Korpuskularstrahlen durch Materie. *Ann. Phys.* **397**, 325–400. https://doi.org/10.1002/andp.19303970303 (1930).
17. Bloch, F. Zur Bremsung rasch bewegter Teilchen beim Durchgang durch Materie. *Ann. Phys.* **408**, 285–320. https://doi.org/10.1002/andp.19334080303 (1933).
18. Bloch, F. Bremsvermögen von Atomen mit mehreren Elektronen. *Z. Phys.* **81**, 363–376. https://doi.org/10.1007/BF01344553 (1933).
19. Thomson, M. *Modern Particle Physics* (Cambridge University Press, 2013).
20. Gcc, the gnu compiler collection—website. https://gcc.gnu.org.
21. Clang: a c language family frontend for llvm—website. https://clang.llvm.org.

22. gcovr—website. https://gcovr.com/en/stable/.
23. Googletest repository. https://github.com/google/googletest.
24. Gitlab website. https://gitlab.com.
25. Doxygen website. https://doxygen.nl.
26. BLAS (basic linear algebra subprograms).
27. LAPACK—linear algebra PACKage (2023).
28. OpenBLAS : An optimized BLAS library (2023).
29. The LAPACKE c interface to LAPACK.
30. Inc, A. Accelerate.
31. Pusz, M. A c++ approach to physical units (2019).
32. Renda, M., Ciubotaru, D. & Banu, C. Betaboltz: A Monte-Carlo simulation tool for gas scattering processes. *Comput. Phys. Commun.* **267**, 108057. https://doi.org/10.1016/j.cpc.2021.108057 (2021).

Acknowledgements

This study was supported by the Romanian Ministry of Research, Innovation and Digitization through the `PN23210104` and `ATLAS CERN-RO` Grants.

Author contributions

E.S.: software, validation, writing—original draft, visualization. D.C.: software, validation, writing—review and editing. M.R.: conceptualization, software, writing—review and editing, supervision. C.A.: resources, writing—review and editing, supervision, project administration, funding acquisition.

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to C.A.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

© The Author(s) 2023